

Week 6 - Lecture 12 - Problem: Algorithms, Recursion and Running Time

ILO

- **KU3:** Design algorithms to solve simple programming problems.
- **CS1:** Structure small programs using iterations, functions, classes, and methods.
- **CS2:** Structure programs into manageable and reusable units.
- **CS3:** Write readable, descriptive, and well-documented program code.
- **CS5:** Express mathematical formulas as programming expressions and algorithms.
- **CS9:** Use standard libraries and follow good programming practices.

Objectives

After completing this problem, you should be able to:

1. Solve the same problem with an iterative and a recursive algorithm.
2. Explain how recursive calls use the call stack, and why a base case is required.
3. Measure running time with `time.perf_counter()` and benchmark a function over many repetitions.
4. Explain best, average, and worst cases, and describe how running time grows with input size (Big O).
5. Compare linear search and binary search, and explain why binary search requires sorted data.
6. Use `enumerate`, tuple unpacking, and argument unpacking (`*args`) to write shorter and clearer code.
7. Use comprehensions to generate test data.

Background

The same job can often be done in more than one way. A **factorial** can be computed with a loop (iterative) or by a function that calls itself (recursive). Both give the same answer, but they work differently and can take different amounts of time.

A recursive function needs:

- a **base case** that stops the recursion, and
- a **recursive case** that calls the function on a smaller problem.

Each call is placed on the call stack and waits until the call it made returns. Too many nested calls cause a `RecursionError`.

The time an algorithm needs may depend on the **magnitude** of the input (for example, `n` in `fac(n)`) or on the **number of items** in the input (for example, the length of a list that is searched). **Big O** notation describes how the running time grows as the input grows:

Notation	Growth	Example
$O(1)$	constant	list index
$O(\log n)$	logarithmic	binary search

Notation	Growth	Example
$O(n)$	linear	linear search
$O(2^n)$	exponential	naive recursive Fibonacci

Running time is also affected by which case occurs:

- **Best case:** the target is found immediately.
- **Worst case:** the target is last, or not present.
- **Average case:** the typical situation.

You will also practice some Python techniques that this problem needs:

```
a, b = b, a + b          # swap and update in one step
for ind, num in enumerate(data): # enumerate gives index and value together
    function(*args)         # unpack a tuple into separate arguments
data = [random.randint(1, 100) for _ in range(100)] # list comprehension
```

Task

Part 1: Factorial two ways

Implement both functions:

```
def fac_iterative(n):
    ...

def fac_recursive(n):
    ...
```

`fac_iterative` must use a `for` loop with `range(2, n + 1)`. `fac_recursive` must have a base case for `n <= 1`.

Check that both give the same result for `n = 0, 1, 5, 10`.

Draw the recursion diagram for `fac_recursive(4)` showing each call and the value it returns. Then call `fac_recursive(5000)` and describe what happens. Why does the iterative version not fail?

Part 2: Fibonacci two ways

Implement:

```
def fib_iterative(n):
    ...
```

```
def fib_recursive(n):  
    ...
```

Use $F(0) = 0$ and $F(1) = 1$. In `fib_iterative`, use sequence unpacking, `a, b = b, a + b`, and use `_` as the loop variable because it is not needed.

Draw the recursion diagram for `fib_recursive(5)`. Which calls are repeated? How many calls does `fib_recursive(5)` make?

Part 3: Measure running time

Measure one call:

```
start = time.perf_counter()  
fac_iterative(60)  
end = time.perf_counter()  
print(f'time taken iterative: {end - start:.6f}')
```

The output of `perf_counter` is in seconds. A single measurement is not reliable, so write a benchmark function that repeats the call and returns the average:

```
def benchmark(function, input, repetitions):  
    ...
```

Use it to compare `fac_iterative` and `fac_recursive` for `n = 60`, then `fib_iterative` and `fib_recursive` for `n = 25` and `n = 30`.

Display the average with an f-string format, for example `f'{time_consumed:.6f}'`.

Part 4: Linear search

Create a list of random test data with a list comprehension:

```
data = [random.randint(1, 100) for _ in range(100)]  
target = 17
```

First write this version, which uses `data.index(number)`:

```
def lin_search(data, target):  
    for number in data:  
        if number == target:  
            print(f'{number} found at index {data.index(number)}')
```

Run it. If `target` appears more than once, why is the same index printed every time? Why is calling `index` also extra work?

Rewrite the function with `enumerate`, so that you get the index and the value together:

```
def lin_search(data, target):
    for ind, num in enumerate(data):
        if num == target:
            print(f'{num} found at index {ind}')
```

Finally, change it to return only the **first** match and not print inside the function. Return `-1` if the target is not found. Explain why best, average, and worst cases make sense for this version, but not for the "print all matches" version.

Part 5: Benchmark with argument unpacking

Search functions take more than one argument, so the benchmark needs to pass several values. Change `benchmark` to receive the arguments as a tuple and unpack them in the call:

```
def benchmark(function, input, repetitions):
    start = time.perf_counter()
    for _ in range(repetitions):
        function(*input)
    end = time.perf_counter()
    return (end - start) / repetitions
```

Call it as:

```
time_consumed = benchmark(lin_search, (data, target), 10)
print(f'{time_consumed:.6f}')
```

Measure the best case (the target at index 0), the worst case (the target is not in the list), and a random target. Repeat with a list that is 10 times and 100 times larger. How does the average time grow?

Part 6: Binary search

Binary search requires a **sorted** list. Implement it:

```
def bin_search(sorted_data, target):
    start, end = 0, len(sorted_data) - 1
    while start <= end:
        mid = (start + end) // 2
        ...
```

Be careful with the midpoint: `start + end // 2` is not the same as `(start + end) // 2`. Explain why. Use only `sorted_data` in the function. Return the index of the target, or `-1` if it is not found.

Create sorted test data with `sorted(...)` or `range`, and benchmark `lin_search` and `bin_search` on the same sorted list for 1,000, 1,000,000 and, if your computer allows it, 100,000,000 items. Look at how much the average time changes for each when the input is 1000 times larger.

Requirements

- Implement `fac_iterative`, `fac_recursive`, `fib_iterative`, and `fib_recursive`, and show that they agree.
- Use sequence unpacking in `fib_iterative` and `_` for an unused loop variable.
- Use `enumerate` in `lin_search`; do not use `list.index` inside it.
- `lin_search` and `bin_search` must return the index (or `-1`) and not print inside the function.
- Use `time.perf_counter()` for time measurement and report seconds.
- `benchmark` must return the **average** time per call and use `function(*input)`.
- Run `bin_search` only on sorted data.
- Use f-strings to format the output.

Discussion Questions

1. What are the base case and recursive case in `fac_recursive`? What happens if the base case is missing?
2. How does the call stack change during `fac_recursive(4)`?
3. Why is `fib_recursive` much slower than `fib_iterative` for large `n`, while `fac_recursive` and `fac_iterative` are close?
4. Why is the average over many repetitions a better measurement than one measurement?
5. What are the best, average, and worst cases of `lin_search` and `bin_search`?
6. Why does binary search need sorted data, and how does sorting change the total cost if you search only once?
7. What does `function(*input)` do, and how is it different from `function(input)`?
8. How do `enumerate` and `list.index` differ in what they cost?

Extension Challenges

1. Add memoization to `fib_recursive` with a dictionary and compare the running time with the original.
2. Write a recursive version of `bin_search` and compare it with the iterative version.
3. Count the number of recursive calls made by `fib_recursive(n)` for `n = 5, 10, 15, 20, 25` and describe how the count grows.
4. Compare `time.perf_counter()` with `time.process_time()` in `benchmark`. When do they give different results?
5. Use `timeit` instead of your own `benchmark` and compare the results.